

TECHNOVA HACK 2026
SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

CampusConnect

Smart Communication & Discussion Portal for College Management

Submitted By

Team Name: WEB CODERS

BY :

NISHANTH P

MOHAMMED AJMAL B

MUKESH PANDIYAN R

PRACHITHA R

Technology Stack

Frontend: React.js, Tailwind CSS

Backend: Node.js, Express.js

Database: MySQL

Version Control: Git & GitHub

Academic Year 2026–2027

CHAPTER 1: INTRODUCTION

1.1 Purpose

CampusConnect is a web-based communication and collaboration platform developed to improve communication within educational institutions. It provides a centralized system for managing announcements, academic resources, discussions, support tickets, notifications, and emergency alerts. This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of the system and serves as a reference for development, testing, and future enhancements.

1.2 Scope

CampusConnect is designed for colleges and universities to simplify communication among students, faculty, and administrators. The system enables users to access announcements, download study materials, participate in discussions, raise support tickets, and receive notifications through a single platform. It reduces dependency on multiple communication channels and improves information accessibility.

1.3 Objectives

The main objectives of CampusConnect are:

- To centralize academic communication.
 - To provide role-based access for students, faculty, and administrators.
 - To manage announcements efficiently.
 - To enable academic resource sharing.
 - To support discussions and collaborative learning.
 - To provide a support ticket management system.
 - To deliver real-time notifications and emergency alerts.
-

1.4 Technologies Used

Component	Technology
Frontend	React.js
Styling	Tailwind CSS
Backend	Node.js & Express.js
Database	MySQL

Component	Technology
-----------	------------

API	REST API
-----	----------

Version Control Git & GitHub

1.5 Users of the System

The system supports three types of users:

- **Administrator:** Manages users, announcements, resources, support tickets, and emergency alerts.
 - **Faculty:** Uploads academic resources, participates in discussions, and views announcements.
 - **Student:** Views announcements, downloads resources, participates in discussions, raises support tickets, and receives notifications.
-

1.6 Advantages

- Centralized communication platform.
- Easy sharing of academic resources.
- Real-time notifications and emergency broadcasts.
- Secure role-based access.
- Improved collaboration between students and faculty.
- Efficient support ticket management.

CHAPTER 2: OVERALL DESCRIPTION

2.1 Product Perspective

CampusConnect is a centralized web-based communication platform developed for educational institutions. It replaces traditional communication methods such as WhatsApp groups, emails, and notice boards by providing a single platform for managing announcements, academic resources, discussions, notifications, and support services. The system improves communication efficiency and ensures that important information reaches the intended users.

2.2 Product Functions

The CampusConnect system provides the following major functions:

- User Authentication with role-based access.
- Dashboard displaying key statistics and recent activities.
- Announcement creation and acknowledgement tracking.
- Resource upload, download, and management.

- Academic discussion forum with replies.
 - Support ticket creation and status management.
 - Notification management.
 - Emergency alert broadcasting.
 - User management for administrators.
-

2.3 User Classes

Administrator

The administrator manages the entire system, including users, announcements, resources, emergency alerts, notifications, and support tickets.

Faculty

Faculty members can upload study materials, participate in discussions, respond to student queries, and access announcements.

Student

Students can view announcements, acknowledge important notices, download academic resources, participate in discussions, raise support tickets, and receive notifications.

2.4 Operating Environment

The application is designed to run on modern web browsers and requires an internet connection.

Client Side

- Windows, Linux, or macOS
- Google Chrome, Microsoft Edge, Mozilla Firefox

Server Side

- Node.js
 - Express.js
 - MySQL Database
-

2.5 Design Constraints

The following constraints were considered during development:

- Internet connectivity is required.
- Users must log in to access the system.
- Role-based permissions restrict unauthorized access.

- MySQL is used as the primary database.
 - The system is designed as a web application.
-

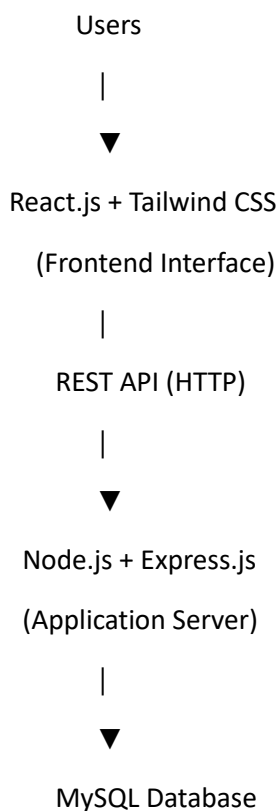
2.6 Assumptions and Dependencies

The system assumes that:

- Users have valid login credentials.
 - The database server is available.
 - A stable internet connection is available.
 - Users access the system through a supported web browser.
-

2.7 System Architecture

CampusConnect follows a three-tier architecture consisting of the presentation layer, application layer, and database layer.



The frontend communicates with the backend using REST APIs, while the backend processes requests, applies business logic, and stores or retrieves data from the MySQL database.

CHAPTER 3: FUNCTIONAL REQUIREMENTS

The CampusConnect system consists of several modules that work together to provide a centralized communication platform for educational institutions.

3.1 User Authentication

Description:

The system provides secure login functionality for administrators, faculty members, and students.

Functional Requirements:

- Users can log in using their registered email and password.
- The system validates user credentials.
- Users are redirected to their respective dashboards based on their roles.
- Unauthorized users are denied access.

3.2 Dashboard

Description:

The dashboard provides an overview of important information and recent activities.

Functional Requirements:

- Display total users.
- Display total announcements.
- Display total resources.
- Display open support tickets.
- Show recent announcements.
- Show recent notifications.

3.3 Announcement Management

Description:

Administrators can manage official announcements for students and faculty.

Functional Requirements:

- Create announcements.
- Edit announcements.
- Delete announcements.
- View announcement details.
- Set announcement priority.
- Select target audience.
- Track acknowledgements.

3.4 Resource Management

Description:

Faculty and administrators can upload and manage academic resources.

Functional Requirements:

- Upload study materials.
 - Download resources.
 - Delete resources.
 - Search resources.
 - View uploaded resources.
-

3.5 Discussion Forum

Description:

Provides a collaborative platform for academic discussions.

Functional Requirements:

- Create discussion topics.
 - View discussions.
 - Reply to discussions.
 - Display discussion history.
-

3.6 Support Desk

Description:

Allows users to report issues and track their resolution.

Functional Requirements:

- Create support tickets.
 - View ticket details.
 - Reply to tickets.
 - Update ticket status.
 - Close resolved tickets.
-

3.7 Notification Management

Description:

Displays system notifications to users.

Functional Requirements:

- Display notifications.
 - Mark notifications as read.
 - Delete notifications.
 - Show unread notification count.
-

3.8 Emergency Alerts

Description:

Allows administrators to broadcast urgent messages to users.

Functional Requirements:

- Create emergency alerts.
 - Select target audience.
 - Assign severity level.
 - View previous alerts.
-

3.9 User Management

Description:

Administrators can manage user accounts within the system.

Functional Requirements:

- Add new users.
 - Update user details.
 - Delete users.
 - Search users.
 - Assign user roles.
-

3.10 Functional Requirement Summary

Module	Description
User Authentication	Secure login with role-based access
Dashboard	Displays system statistics and recent activities

Module	Description
Announcement Management	Create and manage announcements
Resource Management	Upload and download academic resources
Discussion Forum	Academic discussions and replies
Support Desk	Ticket creation and issue tracking
Notification Management	Displays system notifications
Emergency Alerts	Broadcast urgent announcements
User Management	Manage students, faculty, and administrators

CHAPTER 4: NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and performance expectations of the CampusConnect system. These requirements ensure that the application is secure, reliable, efficient, and user-friendly.

4.1 Performance Requirements

- The system should respond to user requests within **2–3 seconds** under normal conditions.
 - Dashboard statistics and announcements should load quickly.
 - The application should support multiple users accessing the system simultaneously without significant performance degradation.
-

4.2 Security Requirements

- Only authenticated users can access the system.
 - Role-based access control ensures users can only access authorized modules.
 - User passwords should be stored securely in encrypted form.
 - Input validation should be performed to prevent invalid or malicious data.
-

4.3 Reliability Requirements

- The system should provide consistent and accurate information.
- Database transactions should maintain data integrity.
- Proper error handling should be implemented to avoid application crashes.
- Regular database backups should be maintained.

4.4 Usability Requirements

- The user interface should be simple, responsive, and easy to navigate.
- Users should be able to perform common tasks with minimal training.
- The application should provide clear error and success messages.

4.5 Scalability Requirements

- The system should support an increasing number of users without major modifications.
- New modules and features should be integrated easily due to the modular architecture.
- The database should support future expansion as institutional data grows.

4.6 Availability Requirements

- The application should be available whenever users need to access academic information.
- Regular maintenance should be scheduled to minimize service interruptions.
- In case of failures, the system should recover with minimal data loss.

4.7 Compatibility Requirements

The application should be compatible with:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Windows, Linux, and macOS operating systems

4.8 Maintainability Requirements

- The source code should be modular and well-structured.
- RESTful APIs should simplify future enhancements.
- Database tables should be normalized for easier maintenance.
- The system should allow easy debugging and updates.

CHAPTER 5: SYSTEM ARCHITECTURE AND DATABASE DESIGN

5.1 System Architecture

CampusConnect follows a **three-tier architecture** consisting of the Presentation Layer, Application Layer, and Database Layer. This architecture ensures better scalability, maintainability, and efficient communication between different system components.

Presentation Layer

The Presentation Layer is developed using **React.js** and **Tailwind CSS**. It provides an interactive and responsive user interface for administrators, faculty members, and students.

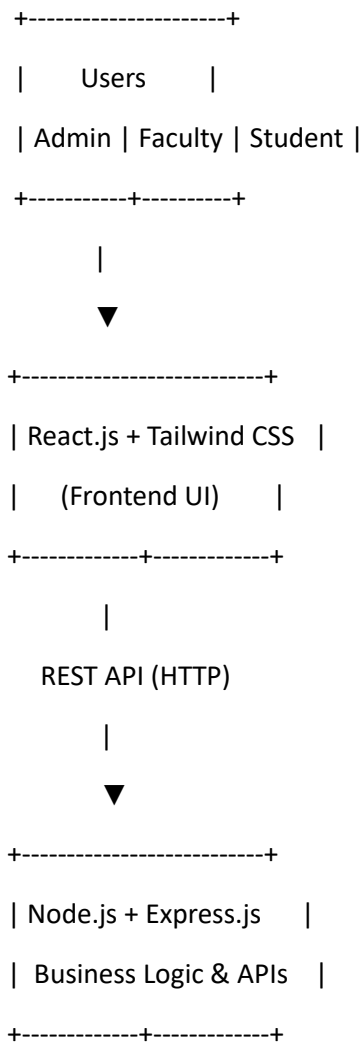
Application Layer

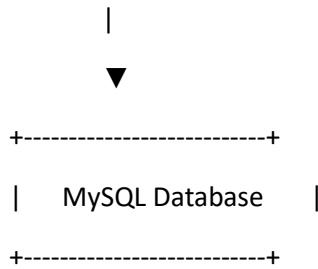
The Application Layer is developed using **Node.js** and **Express.js**. It handles business logic, user authentication, request processing, and communication between the frontend and the database through RESTful APIs.

Database Layer

The Database Layer uses **MySQL** to store and manage application data such as users, announcements, resources, discussions, notifications, and support tickets.

5.2 System Architecture Diagram





5.3 Technology Stack

Layer	Technology
Frontend	React.js
Styling	Tailwind CSS
Backend	Node.js
Framework	Express.js
Database	MySQL
API Communication	REST API
Version Control	Git & GitHub
Deployment	GitHub Pages (Frontend)

5.4 Database Design

The CampusConnect system uses a **MySQL relational database** to store and manage all application data. The database is designed using normalized tables to reduce redundancy and maintain data integrity. Relationships between tables are established using primary and foreign keys.

Main Database Tables

Table Name	Description
Users	Stores administrator, faculty, and student details
Announcements	Stores announcement information
Announcement_Acknowledgements	Tracks announcement acknowledgements
Resources	Stores uploaded academic resources
Subjects	Stores subject information
Student_Subjects	Maps students to their enrolled subjects

Table Name	Description
Discussions	Stores discussion topics
Discussion_Replies	Stores replies to discussions
Notifications	Stores user notifications
Support_Tickets	Stores support requests
Support_Replies	Stores replies to support tickets

5.5 Database Relationships

The major relationships in the database include:

- One administrator can create multiple announcements.
- One announcement can have multiple acknowledgements.
- One faculty member can upload multiple resources.
- One discussion can have multiple replies.
- One support ticket can have multiple replies.
- One student can enroll in multiple subjects.
- One subject can have multiple students.

These relationships ensure efficient data management and maintain consistency across the system.

CHAPTER 6: FUTURE ENHANCEMENTS

Although CampusConnect provides a comprehensive communication and collaboration platform, several enhancements can be implemented in future versions to improve functionality and user experience.

The planned future enhancements include:

- Development of a mobile application for Android and iOS devices.
- Integration with the college ERP system for attendance, timetable, and examination management.
- Push notifications for instant updates on mobile devices.
- AI-powered chatbot to assist users with common queries and navigation.
- Email and SMS notifications for important announcements and emergency alerts.
- Advanced analytics and reporting dashboard for administrators.
- Cloud deployment for improved scalability and availability.
- Integration with video conferencing platforms for online meetings and virtual discussions.

These enhancements will make CampusConnect a more intelligent, scalable, and comprehensive solution for educational institutions.

CHAPTER 7: CONCLUSION

CampusConnect is a centralized web-based communication and discussion portal designed to improve communication and collaboration within educational institutions. The system integrates multiple academic services such as announcements, resource sharing, discussion forums, support ticket management, notifications, emergency broadcasting, and user management into a single platform.

By replacing fragmented communication methods with a unified system, CampusConnect improves information accessibility, enhances collaboration among students and faculty, and reduces administrative workload. The use of modern web technologies such as React.js, Node.js, Express.js, and MySQL ensures that the application is secure, scalable, and easy to maintain.

The project successfully addresses the communication challenges faced by educational institutions and provides a strong foundation for future enhancements and integration with other academic systems.

REFERENCES

1. React Documentation – <https://react.dev>
2. Node.js Documentation – <https://nodejs.org>
3. Express.js Documentation – <https://expressjs.com>
4. MySQL Documentation – <https://dev.mysql.com/doc>
5. Tailwind CSS Documentation – <https://tailwindcss.com/docs>
6. Git Documentation – <https://git-scm.com/doc>
7. GitHub Documentation – <https://docs.github.com>